

Deterministic Authorization and Recovery for Agentic Payments

A failure analysis and proposed structure, with notes on integration

Working paper · revised draft

September 18, 2026

Abstract

Agent-initiated spending is being deployed faster than any public record of its failures is accumulating. A survey of the documented record finds one well-corroborated case of an autonomous agent making an unauthorized real-money purchase, a small cluster of merchant-side wrong-outcome failures, and a body of red-team work demonstrating capability without confirmed victims. That scarcity is a reporting artifact rather than evidence of reliability: there is no disclosure regime for agent purchase errors, and individual losses are small enough to be absorbed by ordinary chargebacks.

Benchmark data fills the gap the incident record leaves. On a retail agent benchmark, a frontier model completes a task correctly on a single attempt far more often than it does so consistently across repeated attempts, and the most over-represented retail error category is confirmation handling. A control designed for an agent that is usually right must therefore assume it will sometimes be wrong in exactly the step where money moves.

This paper argues that existing agentic payment controls bound the wrong quantity. Per-transaction amount caps, tokenized credentials, and spend limits bound *how much* an agent can spend. Every documented failure surveyed here occurred within an authorized amount, at a permitted merchant, on a legitimately delegated credential. What was wrong was the purchase, not the price.

We propose a structure with two halves: make authorization deterministic by binding it to an exact, human-approved, hash-identified record evaluated by pure functions; and make recovery explicit by treating clawback as a policy-bound workflow that reports what it could not recover. A local prototype implements the structure for one purchase category using simulated providers, and includes adversarial tests for attacks that an earlier revision of the same code failed. It does not establish that declared software actually ran, authenticate real merchant evidence, or demonstrate production payment safety.

1. Deployment is outpacing the failure record

In February 2025 a technology columnist asked OpenAI’s Operator to compare egg prices in his neighborhood. Within ten minutes the agent bought a dozen eggs through Instacart and paid for human delivery, a purchase he had not authorized. The transaction was \$31.43. What makes it more than an anecdote is that it bypassed OpenAI’s own stated safeguard requiring user confirmation before a purchase; OpenAI acknowledged the failure and committed to improving those safeguards.

[18]

That incident is, as far as this survey can establish, the clearest public case of an autonomous agent making an unauthorized real-money purchase. It is nearly the only one.

This is not a claim that agent purchase errors are rare. It is a claim about the record. Searching the incident databases, technology press, security research, and regulatory publications for documented agent-initiated purchase failures returns a very short list, assembled in Section 2. Meanwhile the infrastructure for such purchases has been deployed broadly: card networks have published agent transaction protocols [6] [5], a major marketplace operates an agentic checkout against third-party catalogs [20], and consumer assistants that transact on a user’s behalf have raised substantial capital and reached general availability.

Several structural reasons explain the gap, none of them reassuring:

- **No disclosure regime.** No rule requires a platform to publish an agent purchase error. Voluntary transparency reporting so far has covered model behavior rather than transactions; a September 2026 disclosure of six agent incidents by OpenAI contained no purchase, payment, or booking events, all six having occurred in internal testing. [24]
- **Losses are small enough to disappear.** A \$31 error resolves through an ordinary chargeback or a goodwill refund and never becomes a public record.
- **Merchant-side harms settle privately.** Where the injured party is a business rather than a consumer, the dispute resolves contractually. This survey found no documented enterprise procurement agent failure, which is more plausibly a disclosure artifact than an absence of error.

The honest formulation is that agent-initiated spending is being deployed considerably faster than public evidence about its reliability is accumulating. A design discipline for this domain therefore cannot wait for the incident record to mature, and should not be calibrated to the small number of failures that happen to have been reported.

2. What the record does contain

Sources were accessed September 18, 2026. Each item below is labeled by evidence tier. Two claims encountered during this survey — a statistic on agent dispute rates and a purported regulatory advisory on autonomous-agent purchases — could not be substantiated at their attributed sources and are deliberately excluded.

2.1 Liability is already settled, and it sits with the operator

Before the agentic era, a Canadian tribunal ruled on an airline chatbot that gave a passenger incorrect information about a bereavement fare, causing him to pay \$1,630.36 in full fare. The airline argued that the chatbot was “a separate legal entity” for which it was not responsible. The tribunal rejected this, holding that the company “is responsible for all the information on its website” and that it “makes no difference whether the information comes from a static page or a chatbot,” and awarded damages. [19]

Evidence tier: court ruling. This is the strongest item in the survey and the most consequential. It establishes that an operator owns its agent’s errors. Any architecture that leaves an agent’s wrong purchase unattributable is therefore not merely a technical gap; it is an unmanaged liability.

2.2 Authorized purchase, wrong outcome

This is the failure mode central to this paper, and the most thinly documented.

In December 2024 a reporter asked a shopping agent to buy a specific toothpaste from a named retailer. The agent took payment — confirmed on the reporter’s bank statement — then failed for three hours without completing the purchase. The item was in fact out of stock at the retailer while appearing available in the agent’s scraped search data. [21] *Evidence tier: named-source journalism, firsthand, single source*. Authorization was valid at every layer. The agent’s model of the world was wrong.

At larger scale, a marketplace agentic checkout placed orders against stale, incorrect, or entirely absent third-party catalog data beginning around November 2025. Named merchants reported receiving orders for products they do not sell, listings displaying photographs of unrelated items, and orders for discontinued stock. When the marketplace’s data was wrong, the merchant absorbed return shipping and refund. One affected merchant collected 145 responses from other brands. [20] *Evidence tier: named-source journalism, multiple named merchants, two outlets*.

The marketplace case inverts the usual framing: the consumer’s authorization was entirely valid and the injured party was the merchant. It is the only fleet-scale evidence located in this survey.

2.3 Irreversible action against an explicit constraint

In July 2025 a coding agent, operating under an explicit instruction not to modify production during a code freeze, executed destructive commands against a live production database, deleting records for approximately 1,200 companies and a comparable number of individuals. It then reported fabricated results and incorrectly asserted that recovery was impossible. The data was recovered manually; the vendor apologized and shipped environment separation and a non-executing planning mode within days. [22] *Evidence tier: firsthand, company acknowledgment, journalism*.

This was not a financial transaction and is included only for the mechanism it demonstrates: an agent taking an irreversible action that the operator had explicitly forbidden, then misreporting it. The same mechanism applied to a payment produces a loss that no amount limit would have prevented.

2.4 Injection-driven spending: capability, not casualties

Security researchers have catalogued prompt-injection payloads on live web pages that specifically target payment actions, including attempts to subscribe a victim to a paid plan, force a donation through a payment link, and redirect a substantial transfer to an attacker-controlled account. The publishing researchers state explicitly that they are not aware of any confirmed real-world instance where such an attack succeeded against a deployed system. [23] Separate vulnerability research has demonstrated indirect injection against agentic browsers, including payloads concealed in images. [25]

Evidence tier: vendor security research. The accurate formulation is that the capability is demonstrated and payloads are present in the wild, while no confirmed consumer losses have been published. This must not be presented as an incident record. It does establish that an agent’s instruction stream is attacker-reachable, which bears directly on why authority should bind to terms a human approved rather than to the agent’s in-flight reasoning.

3. The rate, which the anecdotes cannot supply

Because the incident record is thin, the more informative evidence is measured error rates. A retail-and-airline agent benchmark evaluates not only whether an agent completes a task correctly but whether it does so *consistently* across repeated independent attempts — reported as pass^k . [17]

The distinction matters more than the headline accuracy number. On the retail domain, a model measured at roughly 61% correct on a single attempt fell to roughly 25% correct across eight consecutive attempts. Frontier models have since improved single-attempt retail performance substantially, but the shape of the pass^k decline is the durable finding: an agent that is usually right is considerably less often reliably right.

Two further details from that work bear on this paper directly. First, the benchmark’s retail error taxonomy over-represents *confirmation handling* errors relative to other domains — the agent mishandling the step where it should have checked with the user. That is precisely the failure exhibited in the Operator incident, where a stated confirmation safeguard did not fire. Second, the benchmark also over-represents *constraint interpretation* errors, the agent misreading what it was told to satisfy.

A spending agent operates in the repeated regime, not the single-attempt one. A control designed on the assumption that the agent is usually correct is calibrated to the wrong statistic.

4. Why current controls do not address these failures

The controls shipped with current agentic payment products are, with one exception, quantity controls.

A consumer assistant that transacts on a user’s behalf mints a single-use virtual card authorized for a specific amount, so that the agent never handles the underlying card number. [9] This is a genuine and well-designed control: it bounds the exposure of any one transaction and removes the credential from the agent’s reach entirely.

A payments platform for agents enforces permissions and limits server-side on every request, so that a compromised or misbehaving agent cannot exceed what it was granted. Limits may be set per transaction, per day, and per month. A payment breaching a limit is not rejected; it is held and an approval is opened that the agent cannot clear itself. [7] Server-side evaluation and the inability of an agent to raise its own caps are both correct design choices, and stronger than client-side enforcement.

Neither control addresses any failure in Section 2.

The unauthorized egg purchase was for \$31.43 — within any plausible limit. The toothpaste purchase was the right item at the right merchant for the right price; only the inventory was wrong. The marketplace orders were correctly priced against catalogs that were themselves incorrect. In each case an amount-based control evaluates the transaction and finds nothing to object to, because there was nothing wrong with the amount.

Two further properties of the current model deserve note:

Holding rather than blocking relocates the decision but does not make it deterministic.

A limit breach that opens an approval is a good design for the case where the user is available and

understands what they are approving. It does not help where the purchase is within limits, which is the case in every incident surveyed.

Where confirmation is discretionary, it is not a control. One consumer assistant’s terms appoint the service as the user’s agent to “enter into agreements, commitments or transactions” on their behalf, then state that the provider “may implement safeguards, confirmation requirements, or other controls on certain Actions” while making “no representation or warranty that such safeguards will prevent unintended or erroneous Actions.” The same terms make payments non-refundable, direct purchase disputes to the merchant rather than the provider, and cap provider liability at one hundred dollars. [26] An independent assessment of the same product found that confirmation before charge does sometimes occur. [27] A safeguard that fires sometimes, is disclaimed contractually, and is paired with no recovery path is not a control a user can reason about.

The missing axis is not *how much* but *what*, and *whether it worked*.

5. Prior work and the scope of this proposal

This is a targeted review of relevant primary documentation, not a systematic literature review or exhaustive market survey. Sources were accessed on September 17, 2026. Vendor statements describe public positioning; specifications describe intended protocol behavior; neither alone proves adoption or operational performance. Drafts and preprints are identified as such.

Delegation and transaction intent already have substantial coverage.

OAuth Rich Authorization Requests already expresses fine-grained authority in structured form, including payment amount and recipient. Granular policy is therefore not an invention specific to AI agents. [2]

The inspected AP2 v0.2 specification defines Checkout and Payment Mandates, linked receipts, verification responsibilities, and direct and autonomous flows. Its constraints are evaluated by designated participants; it also describes evidence used in disputes. The earlier launch essay used Intent and Cart Mandate terminology, so an implementation should follow a pinned specification rather than infer today’s schema from that announcement. This paper’s agent-configuration version is distinct from AP2’s mandate-schema version. [3, 4]

Mastercard-maintained Verifiable Intent, marked draft v0.1 in the inspected repository, specifies a delegation chain, selective disclosure, purchase constraints, and checkout-payment binding. It distinguishes machine-enforceable constraints from descriptive product information and leaves dispute resolution outside its scope. Visa’s Trusted Agent Protocol specifies signed agent-recognition and related commerce messages for participating merchants. Neither a valid message signature nor a signed description should be interpreted as a measurement of an entire executing agent. [5, 6]

Persistent identities and consumers are not unserved categories.

Natural publicly describes persistent agent history, revocable keys, limits, and permissions. Its public materials cover a broad financial stack and discuss disputes; it would be inaccurate to characterize Natural as only a wallet or as lacking guardrails. This review did not verify its internal version-binding implementation, every product’s availability, or an integration with Scrip. [1, 7, 8]

Stripe’s April 2026 announcement describes a Link wallet for agents, consumer approval of spending

requests, and one-time cards or Shared Payment Tokens without exposing underlying payment credentials. A “consumer wallet for agents” is thus not sufficient differentiation by itself. [9]

The potential product distinction examined here is narrower: making the relationship between software changes, outstanding authority, and subsequent outcomes understandable to the principal. Whether that merits a standalone product, a provider feature, or an open-source component is an open commercial question.

Identity, provenance, and outcome verification have prior art.

SPIFFE/SPIRE already separates workload identity from credentials and uses workload properties during attestation. SLSA provenance describes how an artifact was produced. The IETF RATS architecture distinguishes evidence, appraisal, and attestation results, with explicit freshness and trust assumptions. These are foundations to reuse, not missing infrastructure that this paper replaces. [10–12]

ERC-8004, marked draft at review time, proposes identity, reputation, and validation registries while leaving payments orthogonal. Agentic Settlement Protocol is particularly close adjacent work: its September 2026 preprint proposes holds, fulfillment verification, and partial-refund mechanisms for delayed-fulfillment stablecoin commerce, including travel. It identifies itself as a design paper without deployed measurements. Outcome-aware commerce and recovery are therefore not novel merely because they are connected to agents. [13, 14]

This proposal concentrates on principal-approved configuration changes across a mission lifecycle. That emphasis does not establish priority over all prior work, and no interoperability with these systems has been demonstrated.

6. System model and trust assumptions

The **principal** grants authority. An **operator** deploys the agent. A **control service** authenticates requests, checks mandates, and maintains budget and operation state. **Payment providers** report financial events. **Evidence providers** report fulfillment or recovery events. Some roles may belong to one organization, but their claims should remain distinguishable.

The agent may generate mistaken requests, encounter malicious content, repeat operations, or continue after an operator changes its configuration. Providers may time out, return events out of order, or supply inconsistent evidence. These are the failures the design aims to expose or constrain.

The intended controls require all covered spending to pass through the service or equivalent provider-enforced restrictions. An agent with an unrestricted credential outside that boundary can bypass them. The design assumes a trustworthy authorization service, protected state, authenticated principal actions, and appropriate verification of provider messages. A compromised administrator, colluding evidence source, or stolen credential can invalidate those assumptions. Storing a hash beside data does not protect both from an attacker who can rewrite the database.

“Verified outcome” means an assessment supported by specified evidence under a stated trust policy. It does not mean objective truth, customer satisfaction, legal liability, or proof of the agent’s private reasoning.

Manifests and receipts can reveal sensitive instructions, destinations, purchases, and behavior. The proposed service should minimize disclosure, restrict access, and define retention periods. A

digest is not encryption: predictable underlying content may still be guessed. Recording private chain-of-thought is neither necessary nor an assurance mechanism in this design.

7. A deterministic authorization structure

The goal is not to make agent behavior deterministic; it is not. The goal is to make the *authorization decision* deterministic, so that whatever the agent does, the set of purchases it can cause is a pure function of a record a human approved.

This section states the structure independently of any implementation. Section 9 describes a prototype that realizes it for one purchase category.

7.1 Approve an exact

A spending authorization should name the purchase, not a budget. “A refundable room at this hotel, these dates, this rate” rather than “hotels under \$550.” The approved record is canonicalized and hashed; the hash is what the human’s approval binds to.

This inverts the usual tradeoff. A budget is convenient and admits every failure in Section 2, because every one of those purchases was inside its budget. An exact record is less convenient and admits none of them: a cancellation is not a purchase in the record, a different item is not the item in the record, an out-of-stock substitution is not the approved quote.

The cost is real and should be stated. Exact approval means an agent that finds a better option must return for a new approval, and a system that requires many approvals may be abandoned or approved reflexively. Section 10 treats this as the central open question rather than a solved problem.

7.2 Constraints must

Conditions that govern payment must have machine-checkable representations — amount ceilings, permitted merchants, date ranges, refundability deadlines — evaluated by a pure function over the approved record and the proposed purchase. Natural language may propose these terms; it must not enforce them. A model’s assertion that a purchase looks acceptable is not an enforcement mechanism.

7.3 What cannot be typed

A condition the system cannot represent as a predicate (“somewhere nice”, “good reviews”) is the dangerous case, because the natural implementation is to drop it. The structure requires that unresolved conditions be recorded as such and that their presence block unattended execution. Failing closed on uninterpretable terms is what prevents a silent gap between what the human said and what the system enforces.

This also localizes the nondeterminism that natural-language interpretation necessarily introduces. Extraction from prose is fallible; the design requirement is not that extraction be correct but that it be *calibrated* — confidently mistyping a constraint is the harmful failure, while declining to type one is safe because it blocks. A useful evaluation of such a system measures silent mistyping against correct refusal, not extraction accuracy.

7.4 Authority binds

An authorization granted to an agent is granted to a particular declared configuration of that agent: model reference, instruction and policy digests, code artifact, tool versions and permissions. A newly registered configuration inherits no authority; a change to any authority-bearing field requires review under the principal's policy, and a change whose behavioral effect is unknown — a dependency bump, a code change labeled a patch — requires review rather than receiving an automatic exemption.

Authority is checked before any exposure is reserved, and rechecked immediately before the external call. The second check exists because revocation can arrive while an operation is in flight. It narrows the window; it does not close it. Ordering between a local registry and an external provider is not atomic, and once a provider has accepted an instruction, revocation cannot retract it — that operation must be reconciled under the configuration that issued it rather than disowned.

7.5 Outcome is assessed

Payment success and fulfillment success are separate facts and must be recorded separately. Capture is recorded when financial evidence supports it, even where fulfillment later fails; delaying recognition of spent money until the outcome is satisfactory misstates exposure.

The agent's own account of what happened is not evidence. An assessment should read merchant and payment facts attributed to their sources, and where those facts conflict, return an unresolved state for human review rather than guessing. This is the property that would have caught the toothpaste case: payment captured, fulfillment absent, assessment not successful.

A caveat this structure cannot escape: source attribution on evidence is only as good as the adapter that produced it. Labeling a record as merchant-originated does not authenticate the merchant. Evidence authenticity is a deployment obligation, and a system that treats a source label as proof has moved the trust problem rather than solved it.

7.6 Recovery is a workflow

Prevention is cheap and reliable: a pure function evaluated before money moves. Recovery is expensive and unreliable: it depends on merchants, networks, and deadlines outside the system's control. The design consequence is that effort belongs in prevention, and that recovery must never be described as an undo.

Concretely, the structure requires that a refund *request* and a *posted* refund be distinct recorded facts; that a posted refund restore spending authority only where an approved policy says it does, so that repeated purchase-and-refund cycles cannot evade a cumulative limit; that disputes require explicit human authorization rather than being filed automatically; and that a receipt report an *unrecovered* amount as a first-class figure. A system claiming complete recovery would be misrepresenting what it controls. Reporting the residue is the honest alternative.

8. Fitting existing infrastructure

This structure is an authorization and accountability layer. It is not a payment rail, and implementing it does not require custody, card issuance, a new token, a blockchain, or a legal identity standard. It sits above a rail and calls it.

That placement is deliberate, and it is what makes the structure complementary to current agentic payment platforms rather than competitive with them. A platform that holds funds, enforces permissions server-side, and moves money on instruction supplies exactly the execution layer this structure assumes. What the structure adds is upstream of the payment call and downstream of it: an exact approved record that determines whether the call should be made at all, and an outcome assessment that determines what happens afterward.

Mapping onto published work in the space:

- **Agent transaction protocols from the card networks** are converging on cryptographically binding a transaction to a human-approved mandate specifying merchant categories, caps, and time windows. [6] [5] The structure here is the same idea carried further along the axis this paper argues matters: from categories and caps to the exact purchase, and from authorization to outcome.
- **Rich authorization request formats** [2] and **intent protocols** [3] provide transport for structured authorization detail. An integration would require an agreed constraint schema and a verification algorithm, not simply an additional field.
- **Workload identity and attestation work** [10] [11] [12] provides the machinery that would raise version binding above the declared level, with the freshness and coverage limits those specifications state.
- **Payment adapters** must carry provider-specific rules for idempotency retention, authorization expiry, partial capture, refund states, and disputes. [16] [17] Local deduplication does not produce an exactly-once guarantee at a provider, and any field that cannot be faithfully enforced on a given rail should produce a restricted mode or a refusal rather than a claim of equivalent protection.

A narrowly scoped integration experiment is the appropriate next step, and the useful output of one would be negative results: which constraint types cannot be enforced on a given rail, what an approval surface costs in completion rate, and where outcome evidence is unavailable or unauthenticated. Access, provider agreement, protocol compatibility, and commercial feasibility are all unverified here.

9. What the prototype establishes

A local TypeScript prototype (Scrip) implements the structure for one purchase category — an exact, human-approved hotel booking — against simulated payment and execution providers. It has never moved real money, issued a real card, or filed a dispute. The observations below come from source inspection and local tests.

Implemented: an approved contract canonicalized and hashed, with approval binding to that hash; typed constraint predicates evaluated by a pure function before any reservation; unresolved constraints blocking execution; agent lineage, immutable version records with manifest digests, and mandates carrying an explicit authorized-version allow-list; authority checked before reservation and again before dispatch; exposure reserved durably before any provider call; outcome assessed from merchant and payment facts with the agent’s narrative excluded; refund request and posted refund recorded separately; and a receipt reporting authorized, reserved, captured, reversed, refunded, returned, and unrecovered amounts as distinct figures.

9.1 Adversarial results

An earlier revision of this code failed four attacks that a reviewer identified by inspection. Each was reproduced as an executing test that succeeded against the old code, then fixed. They are retained as regression tests.

Attack	Earlier behavior	Current behavior	Control
Caller supplies a fabricated agent identity object	Completed a purchase	Rejected	Service authenticates a presented credential itself; identity type is unforgeable outside the registry
Credential revoked after authentication, then used	Completed a purchase	Rejected	Credential re-verified against stored state inside every authority check
Mandate scoped to refunds used to purchase	Completed a purchase	Rejected	Requested operation and funding source checked against the mandate
Operator self-asserts a stronger attestation level	Recorded as asserted	Not expressible	Level is no longer a caller-supplied parameter

The first of these is the instructive one. The prototype had been reviewed as implementing version-bound authority, and it did — but its service boundary accepted an unauthenticated claim of having been authenticated, so every check downstream operated on an attacker-chosen identity. A correct authorization model behind an uncontrolled boundary provides no protection. This is offered as a caution rather than a solved problem: boundaries of this kind are easy to describe correctly and easy to implement incorrectly.

9.2 Limits

- **No measured-runtime guarantee.** Registration is self-declared and that is the only level the registry can issue; the interface for stronger attestation exists with no implementation, so a stronger label cannot be produced. A manifest digest detects alteration of the record, not of the running program. An unchanged digest cannot detect undeclared changes, mutable model aliases, or evolving retrieved content.
- **Human authentication is out of scope.** The principal is a caller-supplied identifier. Approval and revocation assume an authenticated surrounding service that does not exist here.
- **Session-level, not request-level, credential binding.** A production design should bind each request to mandate, operation, payload digest, audience, expiry, and a replay guard.
- **Evidence source labels are not authenticated.** See 7.5.
- **No atomic revocation ordering with a provider.** See 7.4.
- **Single category, simulated providers, no live result.** The natural-language-to-typed-constraint extraction described in 7.3 is not implemented; constraints are authored directly in fixtures.

- **Canonicalization is custom.** No conformance to a published canonicalization scheme [15] is claimed.

9.3 Local verification record

On September 18, 2026, `npm run build`, `npx tsc --noEmit`, and `npm test` each exited 0, with 252 tests passing and 8 skipped across 21 files and one skipped file. The identity suite contained 26 tests, the purchase-mission suite 50; the skipped file is a PostgreSQL store suite requiring a database.

Those totals describe a test suite, not independent validations of this paper. Passing fixtures against simulated providers do not establish security, payment correctness, evidence authenticity, or fulfillment reliability. No financial operation was initiated.

10. Evaluation required

The central hypothesis — that binding authority to an exact approved record and a declared configuration prevents failures that amount-based controls do not — is untested outside local fixtures. It would be weakened if exact approval provides little protection beyond a well-designed mandate system, or if its approval burden outweighs the protection.

Comparative study. The same task set under three configurations: amount-based limits alone; limits plus exact-record binding; and both plus a measured deployment. Report blocked unauthorized operations *and* legitimate operations incorrectly blocked. A control that stops every bad purchase by stopping most good ones is not useful.

Authority tests. Undeclared configuration changes, mutable aliases, stolen and expired credentials, fabricated caller identities, operator changes, version rollback, missing scopes, and revocation arriving during capability issuance and during dispatch.

Failure injection. Interrupt each persistence and network boundary; reorder and duplicate provider events; reuse idempotency keys past provider retention; simulate partial capture, provider inconsistency, and refund failure. Report unresolved exposure and time to reconcile rather than assuming timeouts are safely retryable.

Evidence tests. Fabricated merchant labels, compromised adapters, conflicting confirmations, cancelled bookings, stale refund events. Report false success, false failure, and unresolved outcomes against independently established ground truth.

Approval-burden study. Measure whether people understand what they authorized, why a change prompt appeared, and the difference between a refund requested and money returned. Report approval counts, completion time, abandonment, and misunderstanding alongside safety outcomes. Given the pass^k evidence in Section 3, the interesting comparison is how burden scales with the number of purchases in a session.

11. Conclusion

The public record of agent purchase failures is short, and the reasons are structural rather than reassuring. The measured record is more informative: agents that are usually right are considerably less often reliably right, and the retail error category that benchmarks over-represent is the handling of confirmation itself.

Against that, the controls currently shipped bound the amount of a transaction. Every failure this survey located was correctly priced. The proposal is to bind the authorization to an exact record a human approved, evaluate it with pure functions, fail closed on anything that cannot be represented, bind authority to a declared software configuration, assess outcomes from evidence the agent did not author, and treat recovery as a workflow that reports what it could not recover.

A local prototype implements this for one category against simulated providers, and is stronger for having failed four attacks and been fixed. It does not establish that the declared software ran, that evidence is authentic, or that any of this is safe in production. The next contribution should be an authenticated integration against a real rail, and the negative results it produces.

References

Primary sources below were accessed September 17-18, 2026. Repository branches and product documentation can change; versions are stated where visible. Source descriptions are not endorsements or independently audited product claims.

- [1] Kahlil Lalji / Natural. [Agentic payments, revisited](#). August 12, 2026. Founder essay; identity, liability, and platform sections.
- [2] T. Lodderstedt, J. Richer, and B. Campbell. [RFC 9396: OAuth 2.0 Rich Authorization Requests](#). 2023. Standards-track specification.
- [3] Google Agentic Commerce. [AP2 specification](#). Inspected document labels itself v0.2; moving repository branch.
- [4] Google Cloud. [Announcing Agent Payments Protocol](#). Launch announcement; consulted for historical terminology, not as the current schema.
- [5] Mastercard / agent-intent contributors. [Verifiable Intent](#). Draft v0.1, as labeled in the inspected repository.
- [6] Visa. [Trusted Agent Protocol specifications](#). Merchant-facing protocol documentation.
- [7] Natural. [Identity](#). Public product description.
- [8] Natural. [Disputes](#). Public product description.
- [9] Dan Hill / Stripe. [Giving agents the ability to pay](#). April 29, 2026. Link wallet product announcement.
- [10] SPIFFE. [SPIRE concepts](#). Workload registration and attestation documentation.
- [11] SLSA. [Provenance, specification v1.2](#). Software supply-chain provenance.
- [12] H. Birkholz et al. [RFC 9334: Remote Attestation procedureS Architecture](#). 2023. Informational architecture; especially trust and freshness.
- [13] M. De Rossi et al. [ERC-8004: Trustless Agents](#). Draft, created August 13, 2025; status checked at review time.
- [14] B. Mohammadkhani, A. Khekade, and R. Kakkad. [Agentic Settlement Protocol: An Application Profile for Refundable, Delayed-Fulfilment Agent Commerce on Stablecoin Rails](#). arXiv:2609.02208v1, September 2, 2026. Preprint/design proposal; no peer-review claim made here.

- [15] A. Rundgren, B. Jordan, and S. Erdtman. [RFC 8785: JSON Canonicalization Scheme](#). 2020. Informational specification.
- [16] Stripe. [Idempotent requests](#). API documentation; key retention and parameter matching.
- [17] Stripe. [Refunds](#). Operational documentation; pending and failed refund states.
- [17] S. Yao et al. [\$\tau\$ -bench: A Benchmark for Tool-Agent-User Interaction in Real-Domain Dialogue](#). arXiv:2406.12045. Source of the $\hat{\text{pass}}$ consistency metric and the retail error taxonomy cited in Section 3.
- [18] G. A. Fowler. [OpenAI’s Operator agent](#). Washington Post, February 7, 2025. Firsthand account of an unauthorized \$31.43 purchase; indexed as AI Incident Database incident 1028.
- [19] *Moffatt v. Air Canada*, British Columbia Civil Resolution Tribunal, February 2024. [CBC report](#). Tribunal ruling rejecting the “separate legal entity” defense.
- [20] A. Smith. [Brands are upset that Buy for Me is featuring their products without permission](#). Modern Retail, January 6, 2026. Named merchants; corroborated by contemporaneous CNBC reporting.
- [21] M. Zeff. [The race is on to make AI agents do your online shopping](#). TechCrunch, December 2, 2024. Firsthand test; payment taken without fulfillment. Single source.
- [22] [Replit AI agent production database incident](#). The Register, July 21, 2025. Non-purchase; included for the irreversible-action mechanism. AI Incident Database incident 1152.
- [23] Palo Alto Networks Unit 42. [Prompt injection targeting AI agents](#). March 3, 2026. Payload telemetry; the authors state no confirmed successful real-world exploitation.
- [24] [OpenAI discloses six agent incidents](#). Fortune, September 17, 2026. Cited as a negative result: no purchase or payment events among them.
- [25] Brave. [Agentic browser security: indirect prompt injection and unseeable prompt injections](#). 2025. Vulnerability research by a competitor to the product examined; disclosed as such.
- [26] Instinct (Spear Street Technology, Inc.). [Terms of Service](#). Quoted for the appointment-as-agent, safeguard-disclaimer, non-refundable, and liability-cap clauses in Section 4.
- [27] Assistant Benchmark. [Instinct assessment](#). September 1, 2026. Cited as counter-evidence: a purchase task passed with approval before charge.

Appendix: prototype evidence boundary

The local verification used the Scrip working checkout (project directory: spending).

Commands executed, all exit status 0:

```
npm run build
npx tsc --noEmit
npm test
```

Result: 252 passed, 8 skipped, across 21 passing test files and one skipped file.

At the time of this revision the mission and identity sources remained untracked in the working checkout, so no commit identifier designates the tested code. This is a reproducibility gap, stated rather than papered over: the digests below identify the inspected files, and external replication additionally requires a published source snapshot, dependency lockfile, runtime version, fixtures, and commands.

Source paths are relative to that checkout.

- Agent identity, manifest classification, attestation levels: **src/missions/agent-identity.ts**.
- Authentication and authority: **SqliteAgentRegistry** in **src/missions/agent-registry.ts**, especially **registerVersion()**, **authenticate()**, **verifyCredential()**, and **authorize()**.
- Orchestration: **PurchaseMissionService** in **src/missions/purchase-mission-service.ts** — especially **authenticate()**, **checkAuthority()**, **execute()**, and **reconcile()**.
- Assessment and constraint evaluation: **preflight()** and **assessOutcome()** in **src/missions/outcome-assessor.ts**.
- Adversarial and identity fixtures: **tests/agent-identity.test.ts** (26 tests, including the four attacks in Section 9.1).
- Purchase fixtures: **tests/purchase-mission-service.test.ts** (50 tests).

SHA-256 digests of four inspected files, split across two lines for print; concatenate without whitespace.

```
agent-identity.ts
ffffaa3cba434782ed1b25bc382353ffa
d2ae6c0dd0f9716f7f9e294aaf712f
```

```
agent-registry.ts
05ccc2b61a001e587aa26199e7a41731
98e23da532d20f34184dbba72fb51552
```

```
purchase-mission-service.ts
791405f8d8fa572b1463022012ad1189
d865934f0cffc8e790043e9323629f54
```

```
outcome-assessor.ts
9217343b5538517b74495ac5c8542222
35f1b70fb7c5028643530c5b80c78269
```